“Real-Time Gait Classification Using IMU Sensor Data and TinyML”

Machine Learning

Course Code: 3CS101CC24

By

Rashi Agarwal (23BEC158)
Jaydev Rathod (23BEC159)
Parth Rathod (23BEC160)
Krish Vaghela (23BEC200)
Aryan Prajapati (23BEE012)

Submitted to:
Dr. Vrajesh Kumar Chawra,
Assistant Professor,
Computer Science & Engineering Department

Department of Computer Science & Engneering
Institute of Technology
NIRMA UNIVERSITY
Ahmedabad 382 481

2026

Abstract

This report describes the design and implementation of a real-time human gait classification system built on Inertial Measurement Unit (IMU) sensor data and TinyML principles.

Raw accelerometer and gyroscope readings are preprocessed, enriched with engineered magnitude features, and segmented into overlapping windows before being fed into a 1D Convolutional Neural Network (CNN).

The system ingests raw triaxial accelerometer and gyroscope readings captured from an IMU sensor positioned on the subject's body. These six-degree-of-freedom (6-DOF) signals inherently contain noise, drift, and orientation-dependent variability, necessitating a robust preprocessing pipeline.

The quantized TFLite model is validated for deployment on the **Seeed studio xiao nRF52840 sense microcontroller** — a widely adopted, platform featuring a ARM cortex-M4 processor, sufficient SRAM, and native support for the TensorFlow Lite for Microcontrollers (TFLM) runtime.

\

1 Dataset and Initial Preprocessing

1.1 Source Data

The input data is stored in a file named walking.csv. Each row represents one time-stamped sensor sample and contains the following raw columns:

Column	Description
`timestamp`	Recording time — not used for classification
`stride_id`	Stride-level identifier — not used for classification
`acc_x, acc_y, acc_z`	Linear acceleration along three orthogonal axes (m/s²)
`gyr_x, gyr_y, gyr_z`	Angular velocity around three orthogonal axes (rad/s)
`class`	Gait label: walk, run, or limp

1.2 Column Removal

Administrative columns (timestamp, stride_id) are removed immediately after loading. They carry no information about motion patterns and would only introduce spurious correlations if retained.

1.3 Label Encoding

The textual class column is converted to integers using scikit-learn's LabelEncoder. The encoder assigns indices in alphabetical order, so the mapping is:

Integer	Class
0	Limp
1	Run
2	Walk

A Counter is printed to verify that the three classes are reasonably balanced in the dataset.

2 Feature Engineering

The six raw sensor axes provide useful directional information, but a classifier also benefits from knowing the overall intensity of motion regardless of device orientation. Two scalar magnitude features are therefore computed and appended.

2.1 Acceleration Magnitude

$$\text{acc_mag} = \sqrt{\text{acc}_x^2 + \text{acc}_y^2 + \text{acc}_z^2}$$

This captures the total linear acceleration at each instant.

Running produces, higher peak values and a distinct rhythmic profile compared to walking, while a limp introduces asymmetry between alternate steps.

2.2 Gyroscope Magnitude

$$\text{gyr_mag} = \sqrt{\text{gyr}_x^2 + \text{gyr}_y^2 + \text{gyr}_z^2}$$

This represents the total rotational speed.

Gait transitions and foot-strike events are often better reflected in angular velocity than in linear acceleration alone.

2.3 Final Feature Vector

The feature matrix X contains eight columns per sample:

```
[acc_x, acc_y, acc_z, gyr_x, gyr_y, gyr_z, acc_mag, gyr_mag]
```

3 About Machine Learning Model

The gait classification system is implemented using a Feedforward Neural Network (Multi-Layer Perceptron – MLP), designed for efficient learning from time-series IMU data while maintaining compatibility with resource-constrained TinyML environments.

The output layer consists of three neurons with softmax activation, corresponding to the gait classes:

- WALK
- RUN
- LIMP

Training is performed using TensorFlow, leveraging its efficient optimization and deployment capabilities. The model uses supervised learning with labeled IMU data and is optimized using gradient-based methods (e.g., Adam optimizer) along with categorical loss functions.

Here, we applied the Window-Based learning. Training is performed using TensorFlow, leveraging its efficient optimization and deployment capabilities.

The model uses supervised learning with labeled IMU data and is optimized using gradient-based methods (e.g., Adam optimizer) along with categorical loss functions.

Overall, the MLP-based model provides a lightweight and computationally efficient solution for gait classification, making it highly suitable for real-time inference on embedded systems and TinyML applications.

Advantages & Disadvantages of Feed-forward Neural Network:

Aspect	Pros (Advantages)	Cons (Limitations)
Model Complexity	Simple architecture, easy to design and implement	Limited capability to model complex temporal dependencies
Computation	Low computational cost, suitable for TinyML and embedded systems	Less expressive compared to CNN/LSTM models
Training Speed	Faster training due to fewer parameters and no sequential processing	May converge to suboptimal solutions for complex patterns
Memory Usage	Requires less memory, ideal for microcontrollers	Cannot efficiently utilize spatial/temporal correlations
Accuracy	Good performance for simple classification tasks (walk/run/limp)	Lower accuracy compared to CNN or RNN for time-series data

4 Windowing

A single IMU sample from one moment in time contains almost no gait information on its own; gait is inherently a temporal pattern that unfolds over roughly one second. The dataset is therefore divided into fixed-length windows that each capture enough context for a reliable classification decision.

4.1 Parameters

Parameter	Value	Rationale
Window Size	100 samples	At 50 Hz sampling, this equals exactly one second of data
Step Size	25 samples	75% overlap between consecutive windows; increase training samples without data leakages

4.2 Purity Filter

Each candidate window spans 100 consecutive rows. The label array for those rows is inspected; if more than one distinct class label appears inside the window, the window straddles a transition between gait types and is discarded.

Only *pure windows* — those where every sample carries the same label — are retained. The majority label of the window becomes its single class target.

This filter prevents the model from being trained on ambiguous mixed-signal inputs, which would degrade both training convergence and test-set accuracy.

The resulting X_w array has shape (N_windows, 100, 8).

5 Training Configuration & Processes

A 1D Convolutional Neural Network is chosen because it can learn local temporal patterns (such as the shape of a foot-strike peak) through learnable filter kernels and progressively builds hierarchical representations of gait rhythm.

5.1 Hyperparameters

Hyperparameter	Value
Optimizer	Adam
Loss Function	Sparse Categorical Crossentropy
Maximum epochs	40
Batch size	32

Adam is selected for its adaptive per-parameter learning rates, which typically converge faster than SGD on time-series tasks. Sparse Categorical Crossentropy is used instead of Categorical Crossentropy because the labels are integer-encoded rather than one-hot vectors.

5.2 Early Stopping

An EarlyStopping callback monitors val_loss with a patience of 5 epochs. If validation loss does not improve for five consecutive epochs, training is halted and the model is automatically rolled back to the epoch with the lowest validation loss via restore_best_weights=True. This prevents wasted computation and guards against overfitting to the training set.

Training may therefore terminate before epoch 40 if convergence is achieved earlier.

```
early_stop = tf.keras.callbacks.EarlyStopping
(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)
```

6 TFLite Conversion

6.1 Default Optimization

The first conversion pass applies TensorFlow Lite's default optimisation strategy, which applies post-training float16 weight compression and operator fusion:

```
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_model = converter.convert()
```

The resulting gait_model.tflite file is significantly smaller than the original Keras saved model and runs faster on devices without GPU support.

6.2 INT8 Full Quantisation

A second, more aggressive conversion quantises weights and activations to 8-bit integers. This requires a representative dataset: a generator that yields 100 randomly sampled inference-shaped inputs (1, 100, 8) is provided so the converter can calibrate quantisation ranges for each layer.

6.3 Deployment Preparation

Once the .tflite binary is produced, it is converted into a C byte array using the xxd utility:

```
xxd -i gait_model.tflite > model.h
```

The resulting model.h header file contains the model as a const unsigned char array and a size constant. This file is included directly in firmware projects (e.g., Arduino sketches for ESP32) where the TensorFlow Lite for Microcontrollers runtime reads the array, allocates tensor arenas, and runs inference.

Conclusion

This project demonstrates a viable end-to-end pipeline for embedded gait classification. Starting from raw CSV IMU recordings, the system applies purposeful preprocessing (column removal, label encoding), feature engineering (magnitude computation), temporal segmentation (overlapping pure windows), and stochastic data augmentation to produce a rich training corpus.

Post-training quantisation reduces the model to a deployable TFLite binary, and a C header file bridges the gap to microcontroller firmware. The approach is practical, resource-efficient, and extendable to additional gait categories or alternative sensor placements.